

# Design and Analysis of Algorithms

## Lecture 10: Spanning Tree and MST



# Excerpts from the previous lectures

- Problems related to Graph:

- ✓ **Graph Traversal**

- ✓ Spanning Tree

- ✓ Minimum Spanning Tree

- ✓ Shortest Path





# Table of Contents

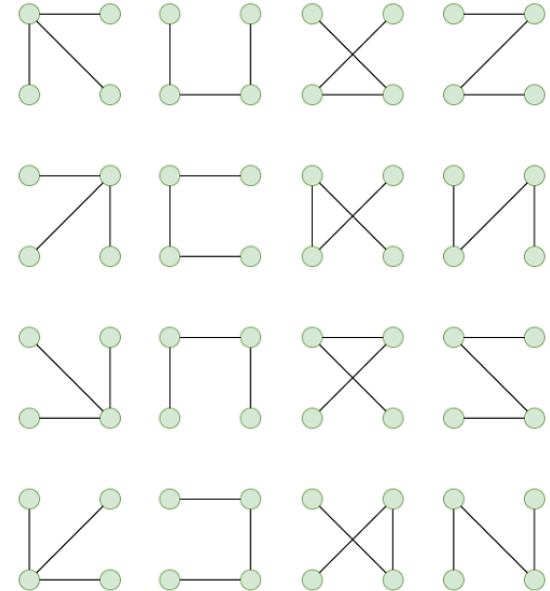
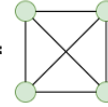
- Problems related to Graph:
  - ✓ Spanning Tree
  - ✓ Minimum Spanning Tree
  - ✓ Shortest Path
- Computational Complexity



# Spanning Tree

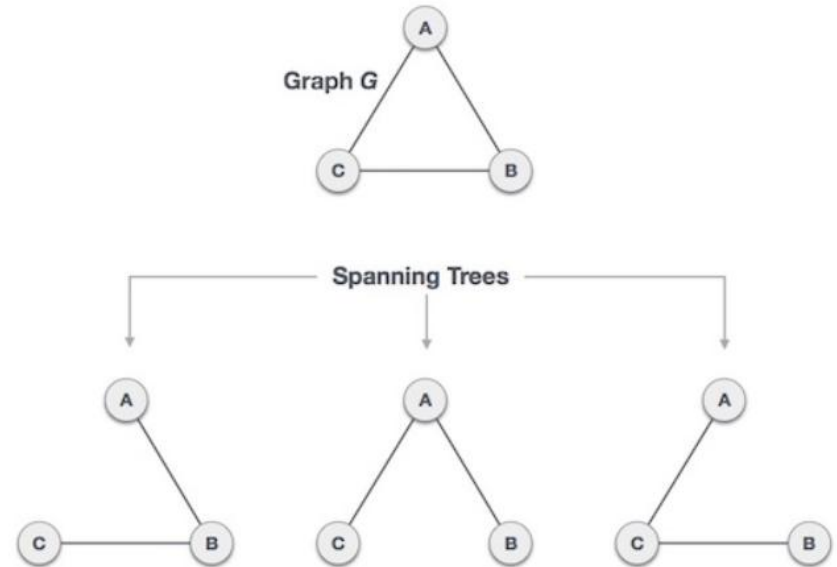
- A spanning tree is a subset of Graph  $G$ , such that all the vertices are connected using minimum possible number of edges.
- Hence, a spanning tree does not have cycles, and a graph may have more than one spanning tree.

Graph =



# Spanning Tree

- By this definition, we can draw a conclusion that every connected and undirected Graph  $G$  has at least one spanning tree. A disconnected graph does not have any spanning tree, as it cannot be spanned to all its vertices.
- A complete undirected graph can have maximum  $n^{n-2}$  number of spanning trees, where  $n$  is the number of nodes. In the addressed example,  $n$  is **3**, hence  $3^{3-2} = 3$  spanning trees are possible.
- We can construct a spanning tree for a complete graph by removing  $E - N + 1$  edges, where  $E$  is the number of Edges and  $N$  is the number of vertices.



# General Properties of Spanning Tree

- We now understand that one graph can have more than one spanning tree. Following are a few properties of the spanning tree connected to graph  $G$ .
  - ✓ A connected graph  $G$  can have more than one spanning tree.
  - ✓ All possible spanning trees of graph  $G$ , have the same number of edges and vertices.
  - ✓ The spanning tree does not have any cycle (loops).
  - ✓ Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**.
  - ✓ Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**.

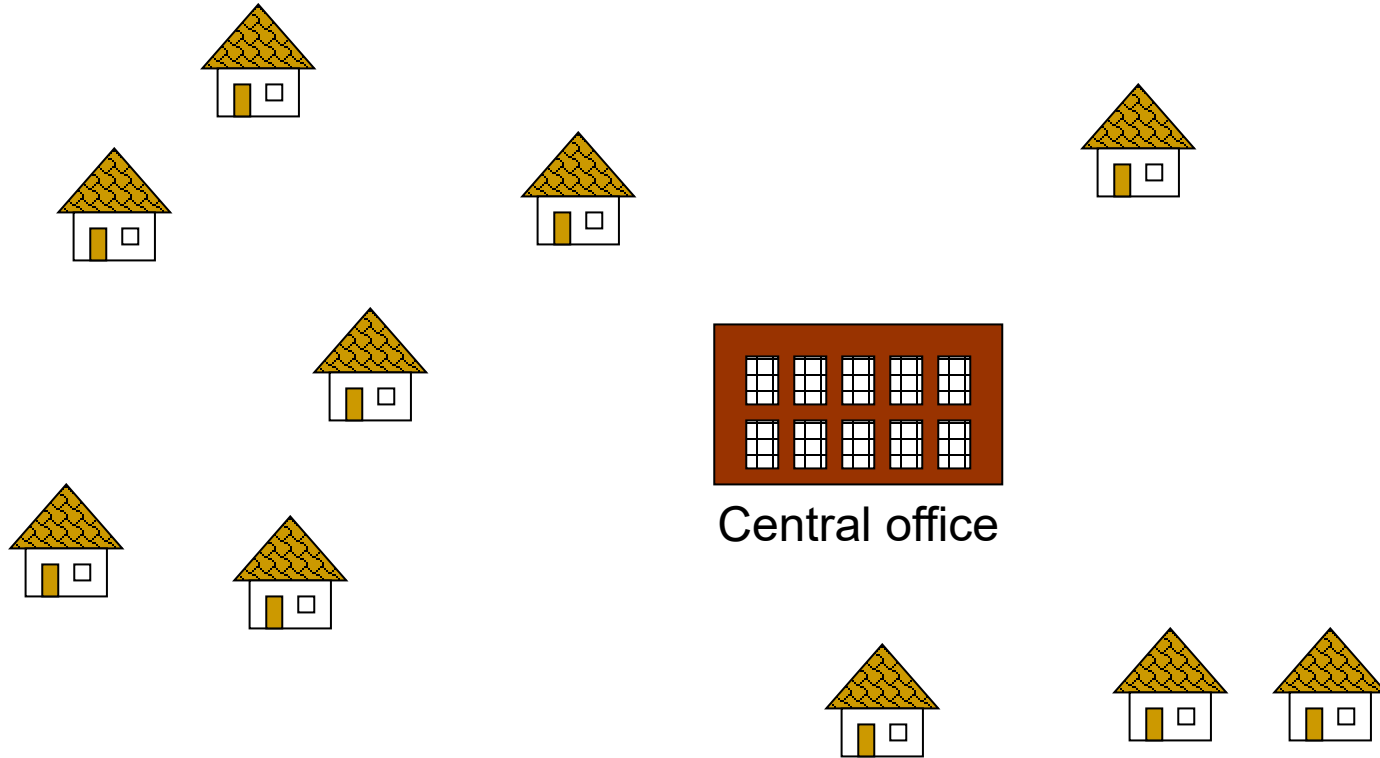


# Application of Spanning Tree

- Spanning tree is basically used to find a minimum path to connect all nodes in a graph. Common application of spanning trees are:
  - ✓ **Civil Network Planning**
  - ✓ **Computer Network Routing Protocol**
- Let us understand this through a small example. Consider, city network as a huge graph and now plans to deploy telephone lines in such a way that in minimum lines we can connect to all city nodes. This is where the spanning tree comes into picture.

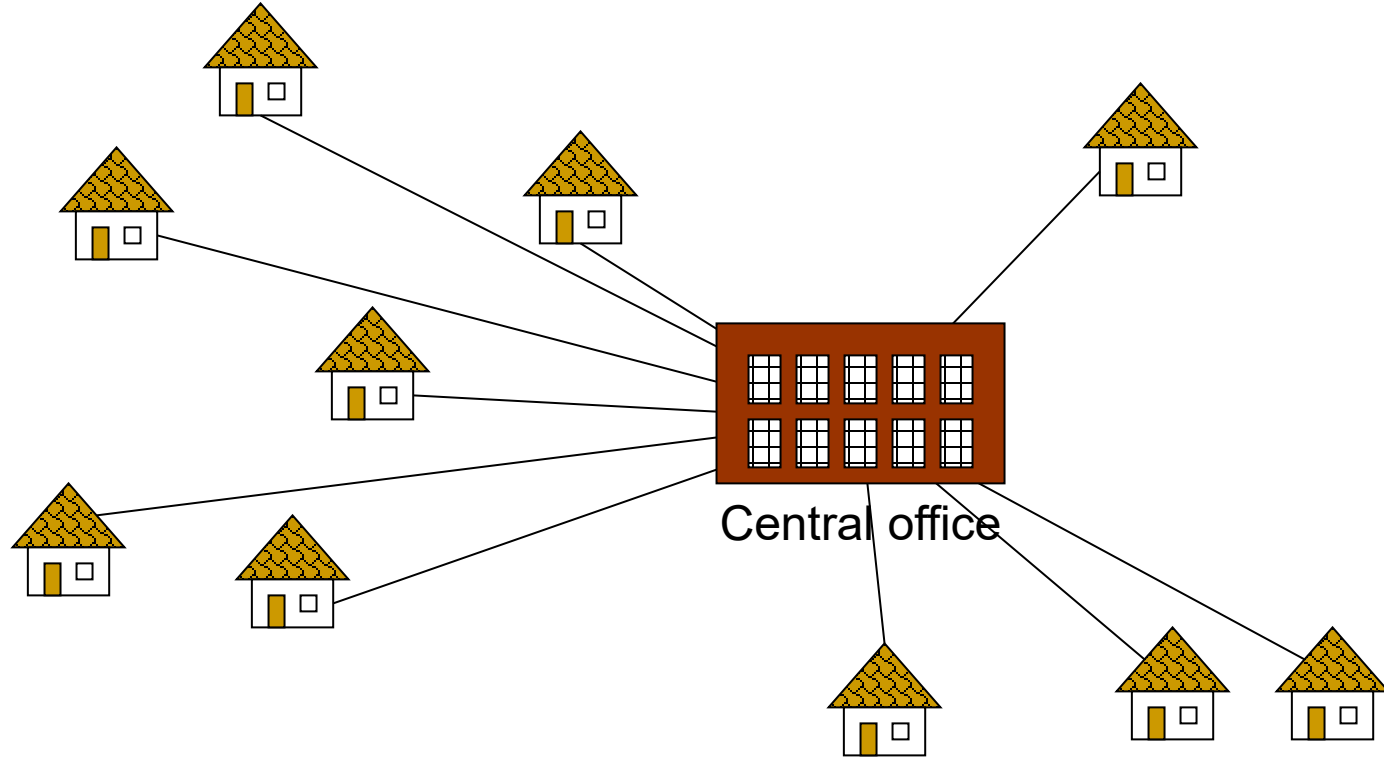


# Problem: Laying Telephone Wire





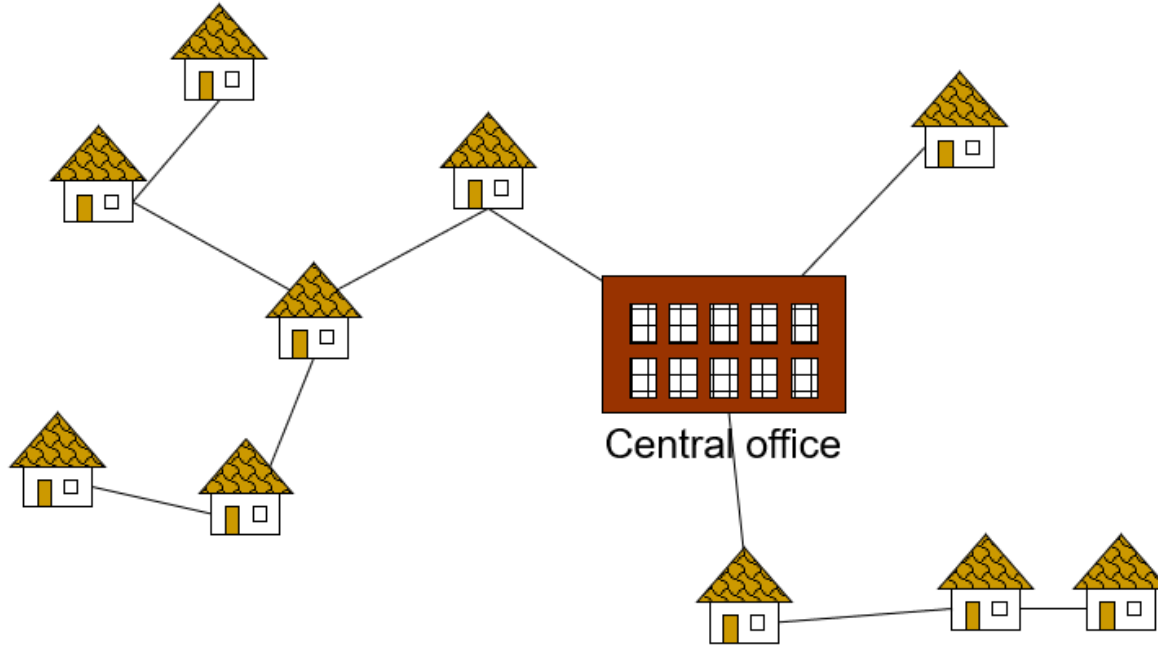
# Wiring: Naïve Approach



**Expensive!**



# Wiring: Better Approach



Minimize the total length of wire connecting the customers



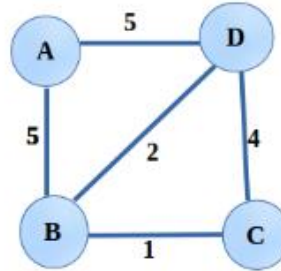
# Minimum Spanning Tree (MST)

- In a weighted graph, a minimum spanning tree is a spanning tree that has minimum weight than all other spanning trees of the same graph.
- In real-world situations, this weight can be measured as distance, congestion, traffic load or any arbitrary value denoted to the edges.

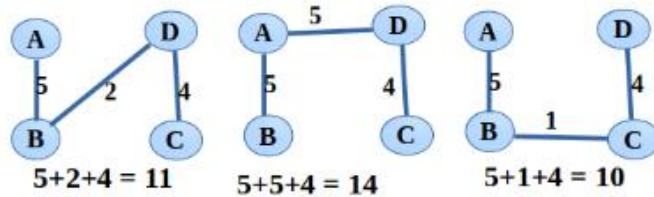


# Minimum Spanning Tree (MST)

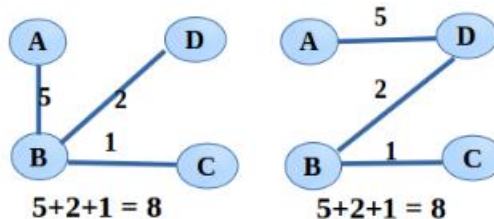
**Connected Graph**



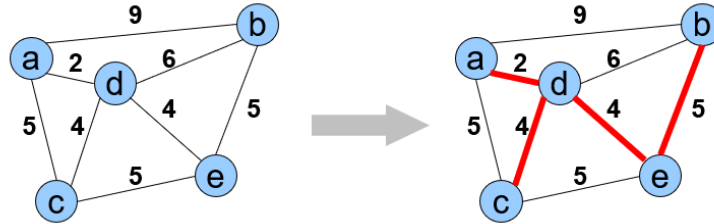
**Spanning Tree**



**Minimum Spanning Tree**



# How can we generate a MST?



**The two widely used famous algorithms are:**

1. Kruskal's algorithm (To study this algorithm, you can visit this website: [https://www.tutorialspoint.com/data\\_structures\\_algorithms/kruskals\\_spanning\\_tree\\_algorithm.htm](https://www.tutorialspoint.com/data_structures_algorithms/kruskals_spanning_tree_algorithm.htm)) (Out of the course scope).
2. Prim's algorithm (To study this algorithm, you can visit this website: [https://www.tutorialspoint.com/data\\_structures\\_algorithms/prims\\_spanning\\_tree\\_algorithm.htm](https://www.tutorialspoint.com/data_structures_algorithms/prims_spanning_tree_algorithm.htm)) (Out of the course scope).

Both algorithms are of greedy approach and give the same output.



# Shortest path problem

- The shortest path problem is about finding a path between vertices in a graph such that the total sum of the edges weights is minimum.
- Several well-known algorithms exist for solving this problem and its variants : such as Dijkstra's algorithm.
- To study Dijkstra's algorithm, you can visit this website:  
[https://www.tutorialspoint.com/data\\_structures\\_algorithms/dijkstras\\_shortest\\_path\\_algorithm.htm](https://www.tutorialspoint.com/data_structures_algorithms/dijkstras_shortest_path_algorithm.htm) (Out of the course scope).



# What is an efficient algorithm?

How about  $O(n \log n)$ ?

$O(n^2)$  ?

$O(n^{10})$  ?

polynomial time

$O(n^c)$  for some  
constant  $c$

---

$O(n^{\log n})$  ?

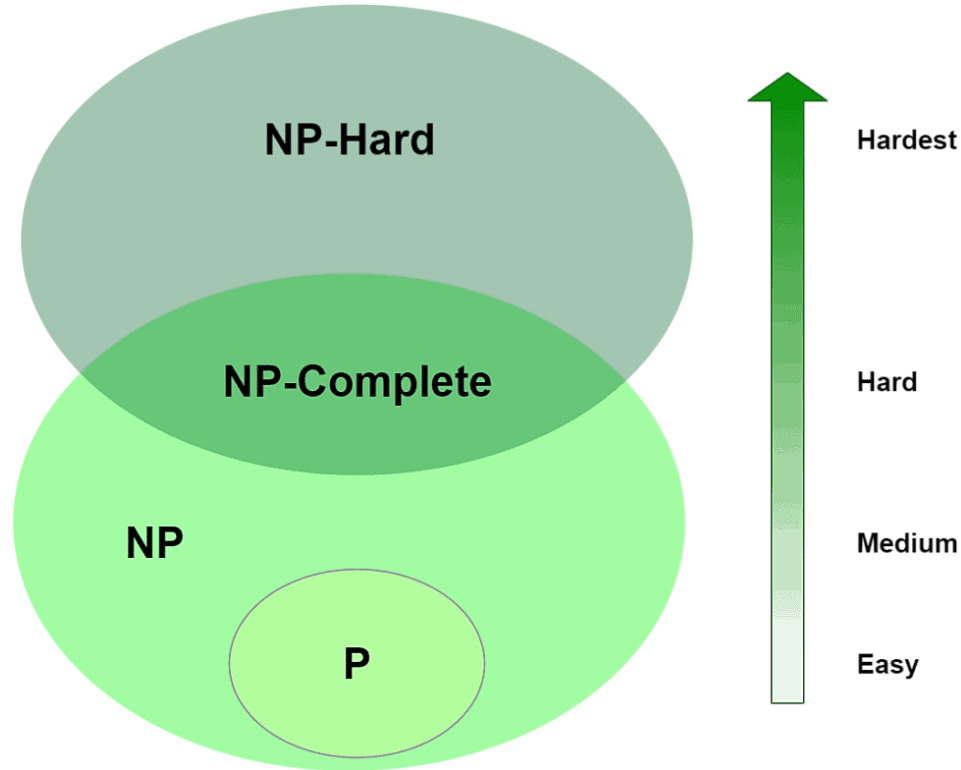
$O(2^n)$  ?

$O(n!)$  ?

non-polynomial time



# Computational Complexity measures





# P (Polynomial) problems

- P problems refer to problems where an algorithm would take a polynomial amount of time to solve, or where Big-O is a polynomial (such as  $O(n)$ ,  $O(n^2)$ , etc).
- These are problems that would be considered easy to solve and thus do not generally have immense run times.
- **Examples of P Problems:**
  - ✓ Insertion sort
  - ✓ Merge sort
  - ✓ Linear search
  - ✓ Matrix multiplication
  - ✓ Finding minimum and maximum elements from the array



# NP (Non-deterministic Polynomial) Problems

- A problem is in NP if there exists a polynomial-time verification algorithm, regardless of the time complexity required to find the solution.
- The solution-finding process may have exponential or factorial time complexity (such as  $O(2^n)$ ,  $O(n!)$ , or  $O(k^n)$ ), but the verification of any candidate solution must run in polynomial time.
- This defines the fundamental asymmetry between solution discovery (potentially intractable) and solution verification (tractable).
- Examples of NP Problems with Their Verification Complexities: 0/1 Knapsack Problem — verification:  $O(n)$  | solution-finding:  $O(2^n)$ .



# NP-Hard and NP-Complete Problems

- **NP-Hard:**

- ✓ **Super Hard Problems** – At least as difficult as the hardest NP problems.
- ✓ **No Easy Checking Needed** – Don't have to be quickly verifiable.

- **NP-Complete:**

- ✓ **Hardest Checkable Problems** – The most difficult problems that are still easy to verify.
- ✓ **In Both Categories** – Must be both NP (easy to check) AND NP-Hard (super hard).



# Any Question?

